HS BO Hochschule Bochum	Hochschule Bochum Bochum University of Applied Sciences

Prüfungsform:	**Klausur**
Prüfungsfach:	**Objektorientierte Programmierung, Objektorientierte Programmiertechniken für Bachelor Informatik und Bachelor Wirtschaftsinformatik**
Semester:	**SoSe 2023**
Datum:	**20.09.2023**
Prüfer:	**Prof. Dr. Ursula Oesing**
Bearbeitungsdauer:	**120 Minuten**
Hilfsmittel:	**Es sind keine eigenen Hilfsmittel zugelassen.**

Kandidat / Kandidatin:

Vorname:	.....
Nachname:	.....
Matrikelnummer:	.....
Unterschrift:	.....

Punkte: Es können 120 Punkte erreicht werden.

Hinweise:

Die 120 Punkte werden in Prozentanteile umgerechnet. Bitte schreiben Sie alle Antworten auf die Aufgabenblätter. Wenn der gegebene Platz unter der Fragestellung nicht ausreicht, können Sie auch die Rückseiten der Aufgabenblätter benutzen. Bitte kennzeichnen Sie jeweils eindeutig, zu welcher Frage ihre Antwort gehört! Unleserliche Bestandteile und Bestandteile auf selbst mitgebrachtem Papier werden nicht gewertet.

Mit der Annahme der Aufgabenstellung erklären Sie sich gesundheitlich in der Lage, die Klausur mitzuschreiben. Bei Täuschungsversuchen wird die Klausur mit 5.0 bewertet.

1. Aufgabe: Versionierung / Versionsverwaltung (5 Punkte)

Erläutern Sie kurz (2-3 Sätze): Was versteht man im Zusammenhang mit Versionsverwaltung unter *Einchecken mit einem Konflikt?*

Basisanwendung

Für die folgenden Aufgaben ist eine Anwendung zur Verwaltung von Sportvereinen als Basis gegeben, siehe die folgenden Seiten.

Klasse Sportverein

```
public class Sportverein {

  private String name;
  private int anzahlMitglieder;
  private int anzahlAktiveMitglieder;

  // Standard-Konstruktor, welcher die Attribute mit Werten seiner
  // Parameter belegt und get- und set-Methoden zu den Attributen

  @Override
  public String toString() {
        String ergebnis = this.getName() + "\n"
              + this.getAnzahlMitglieder() + "\n"
              + this.getAnzahlAktiveMitglieder();
        return ergebnis;
  }
}
```

Klasse Main

```
public class Main {

  public static void main(String[] args) {
        SportvereineControl sportvereineControl
              = new SportvereineControl();
        sportvereineControl.fuelleSportvereine();
        sportvereineControl.zeigeSportvereineAn();
  }
}
```

Klasse SportvereineModel

```
public class SportvereineModel {

  private Sportverein[] sportvereine = new Sportverein[2];

  public void fuelleSportvereine() {
        this.sportvereine[0]
              = new Sportverein("Vfl Bochum", 24162, 19456);
        this.sportvereine[1]
              = new Sportverein("FC Schalke 04", 173523, 150432);
  }

  public String gibSportvereineAus() {
        String ergebnis = "";
        for(Sportverein v : this.sportvereine) {
              ergebnis = ergebnis + v.toString() + "\n\n";
        }
        return ergebnis;
  }
}
```

Klasse SportvereineControl

```
public class SportvereineControl {

  private SportvereineModel sportvereineModel;
  private SportvereineView sportvereineView;

  public SportvereineControl() {
        this.sportvereineModel = new SportvereineModel();
        this.sportvereineView
              = new  SportvereineView(this.sportvereineModel, this);
  }
```

```
    public void fuelleSportvereine() {
          this.sportvereineModel.fuelleSportvereine();
    }

    public void zeigeSportvereineAn() {
          this.sportvereineView.zeigeSportvereineAn();
    }
}
```

Klasse SportvereineView

```
import javax.swing.JOptionPane;

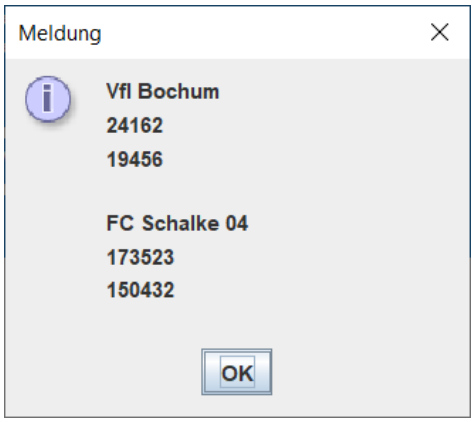
public class SportvereineView {

    SportvereineModel sportvereineModel;
    SportvereineControl sportvereineControl;

    public SportvereineView(SportvereineModel sportvereineModel,
          SportvereineControl sportvereineControl) {
          this.sportvereineModel = sportvereineModel;
          this.sportvereineControl = sportvereineControl;
    }

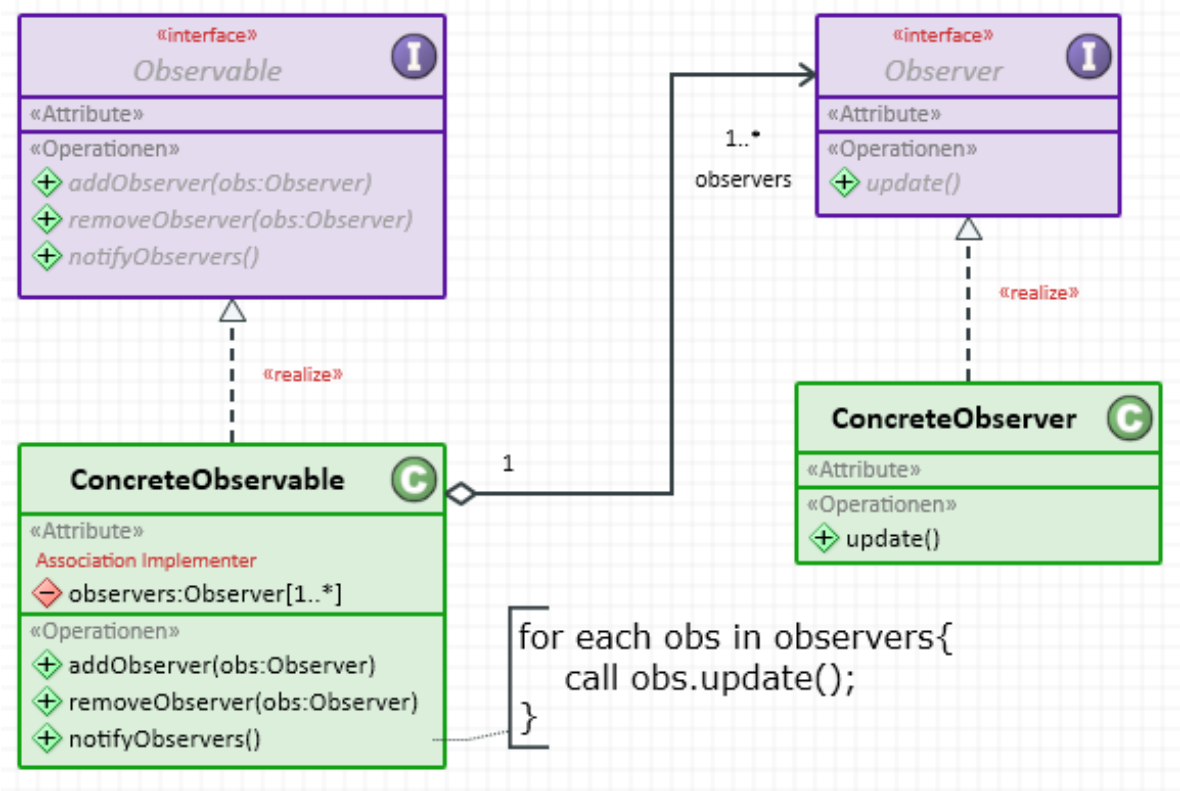
    public void zeigeSportvereineAn() {
          JOptionPane.showMessageDialog(null,
                this.sportvereineModel.gibSportvereineAus());
    }
}
```

Startet man die Anwendung, so erhält man das folgende Ausgabefenster.



2. Aufgabe: Design Patterns (40 Punkte)

Gegeben ist die Basisanwendung, die Anwendung zur Verwaltung von Sport-vereinen. Diese Aufgabe führen Sie auf der Basis dieser Anwendung aus.
Gegeben ist weiterhin ein UML-Klassendiagramm zum Entwurfsmuster **Observer.**



Karl Eilebrecht, Gernot Starke, Patterns kompakt

Geben Sie den Quellcode an, der aus dem UML-Klassendiagramm zu dem Pattern *Observer* hervorgeht bzw. ergänzen Sie die vorliegenden Klassen der Basis-anwendung, so dass diese das Observer Pattern umsetzt:
Welche Klasse wird zum *ConcreteObserver*, welche zum ConcreteObservable?
Übernehmen Sie also nur die Namen der Interfaces *Observer, Observable* nicht die der Klassen *ConcreteObserver und ConcreteObservable* .
Geben Sie den Quellcode der neuen Methoden aus Ihrer *ConcreteObserver*-Klasse und Ihrer *ConcreteObservable*-Klasse an. Die bisherige Funktionalität soll sich insofern ändern, dass immer dann, wenn die Methode *fuelleSportvereine* der Klasse *SportvereineModel* aufgerufen wird, automatisch die Anzeige in der Konsole erfolgt. Aus der Klasse *Main* entfällt daher in der *main*-Methode die Zeile *sportvereineControl.zeigeSprotvereineAn()*.

3. Aufgabe: Vertiefung Java (30 Punkte)

Gegeben ist die Basisanwendung zur Verwaltung von Sportvereinen und eine weitere Klasse *NachhaltigerSportverein*. Packages und Importe werden nicht berücksichtigt.

```
public class NachhaltigerSportverein extends Sportverein{

  private boolean setztSolarthermieEin;
  private boolean setztPhotovoltaikEin;

  // Konstruktor mit Parametern und get- und is-Methoden

  @Override
  public String toString() {
        String ergebnis = super.toString() + "\n"
              + this.isSetztSolarthermieEin() + "\n"
              + this.isSetztPhotovoltaikEin();
        return ergebnis;
  }
}
```

a) Generics, for each

Geben Sie den Quellcode an, um in der Klasse *SportvereineModel* das Attribut *sportvereine* vom Typ *Sportverein[ ]* zu ersetzen durch ein Objekt vom Typ *ArrayList* unter Verwendung von Generics. Geben Sie weiterhin den neuen Quellcode der Methode *fuelleSportvereine* der Klasse *SportvereineModel* an.

b) Generics mit Typeinschränkungen

Geben Sie den Quellcode einer Klasse *SportvereinKonsole* an, welche ein Generic *T* enthält, für welches nur ein Objekt vom Typ *Sportverein* oder einer Ableitung von *Sportverein* eingesetzt werden kann. Es gibt ein privates Attribut *sportvereinBasis* vom Typ *T*, einen Konstruktor, welcher das Attribut mit einem Wert belegt, und eine Methode *gibAttributeInKonsoleAus* ohne Parameter und Rückgabewert. Diese schreibt die Attribute von *sportvereinBasis* in die Konsole, siehe unten. (*SportvereinKonsole* soll beispielsweise für Testausgaben genutzt werden.)

Erstellen Sie in der Klasse *SportvereineControl* folgendermaßen eine Methode *gibSportvereineInKonsoleAus.* Diese hat keine Parameter und keine Rückgabewerte. Erstellen Sie innerhalb dieser Methode ein Objekt *sp1* vom Typ *Sportverein* mit den Werten *Mustersportverein1, 654321, 500000* und ein Objekt *sp2* vom Typ *NachhaltigerSportverein* mit den Werten *Mustersportverein2, 123456, 120000, true, true* und zwei Objekte *sp1Konsole* und *sp2Konsole* vom Typ *SportvereinKonsole* unter Verwendung der Objekte *sp1* und *sp2*. Beim Aufruf dieser Methode (z.B. mittels *sportvereineControl.gibSportvereineInKonsoleAus();* in der *main-Methode* der Klasse *Main*) erscheint die folgende Konsolenausgabe.

```
Die Werte der Attribute von Mustersportverein1 sind die folgenden:
Mustersportverein1
654321
500000
Die Werte der Attribute von Mustersportverein2 sind die folgenden:
Mustersportverein2
123456
120000
true
true
```

4. Aufgabe: JUnit 5 und Zustandsbasierter Test (25 Punkte)

Gegeben ist die Basisanwendung zur Verwaltung von Sportvereinen.

JUnit 5

Ergänzen Sie den vorliegenden Quellcode eines JUnit Tests. Der Konstruktor der Klasse *Sportverein* soll durch diese JUnit Testklasse daraufhin getestet werden, ob die Anzahl der Mitglieder und die Anzahl der aktiven Mitglieder richtig gesetzt werden. Das soll mit einem Testfall innerhalb einer Testmethode, die *Supplier<String>* verwendet, getestet werden. Weiterhin soll der Konstruktor daraufhin getestet werden, ob bei der Eingabe einer Anzahl aktiver Mitglieder, die größer ist als die Anzahl der Mitglieder eine *IllegalArgumentException* mit der Meldung *Die Anzahl der aktiven Mitglieder darf nicht größer sein als die Anzahl der Mitglieder.* geworfen wird. Benutzen Sie für das Testen aus der Klasse *Assertions*
assertTrue(Boolean condition, Supplier<String> messageSupplier) und
assertEquals(String actual, String expected) und
<T> assertThrows(Class<T> clazz, Executable executable).

```
import static org.junit.jupiter.api.Assertions.*;
import  org.junit.jupiter.api.*;

public class SportvereinTest{

  @Test
  public void testSportverein() throws Exception{
       // Ergaenzen!!!









  }

  @Test
  public void testSportvereinException() throws Exception{
       // Ergaenzen!!!





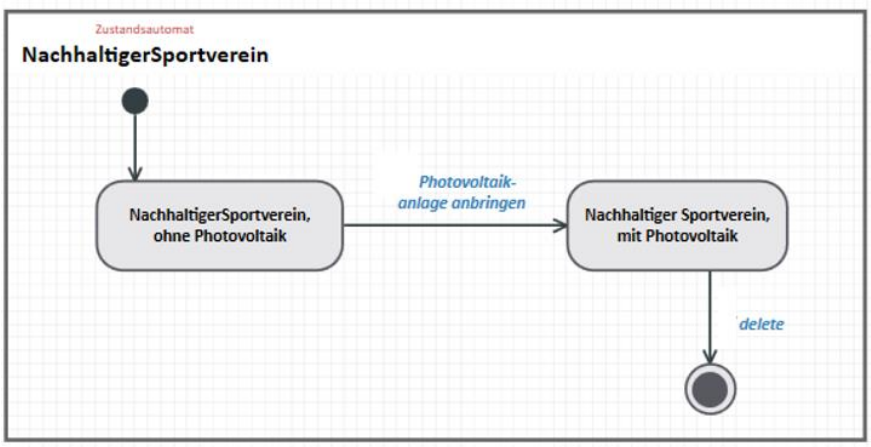




  }
}
```

Zustandsbasierter Test

Gegeben ist ein UML Zustandsautomat, welcher die Zustände eines Objekts vom Typ *NachhaltigerSportverein* der Aufgabe 3 bezüglich der Eigenschaft *Photovoltaik* modelliert. Geben Sie den Übergangsbaum an.
Geben Sie zwei Testfälle inklusive Sollergebnis an, die sich aus dem Übergangsbaum ergeben.



5. Aufgabe: SOLID und Clean Code (20 Punkte)

Gegeben ist die Anwendung zur Verwaltung von Sportvereinen inklusive der Klasse *NachhaltigerSportverein* aus der 3. Aufgabe.

a) Erläutern Sie das *Liskov Substitution Principle.* Nennen Sie ein Beispiel unter Verwendung der Klassen *Sportverein* und *NachhaltigerSportverein*.

b) Wieso trägt das *Liskov Substitution Principle* zu einer höheren Wartbarkeit und Lebensdauer von Software und zu einer Reduktion der Fehleranfälligkeit bei (2-3 Sätze)?

c) Nennen und erläutern Sie kurz zwei der 4 Werte aus Clean Code.

d) Erläutern Sie kurz das Prinzip *Favour Composition over Inheritance (FCoI)*, welches dem roten Grad aus Clean Code zugeordnet ist. Was ist mit *Composition* gemeint, was mit *Inheritance*? Ist das Prinzip bereits in der Verwaltung zu Sportvereinen umgesetzt worden? Begründen Sie kurz Ihre Antwort.